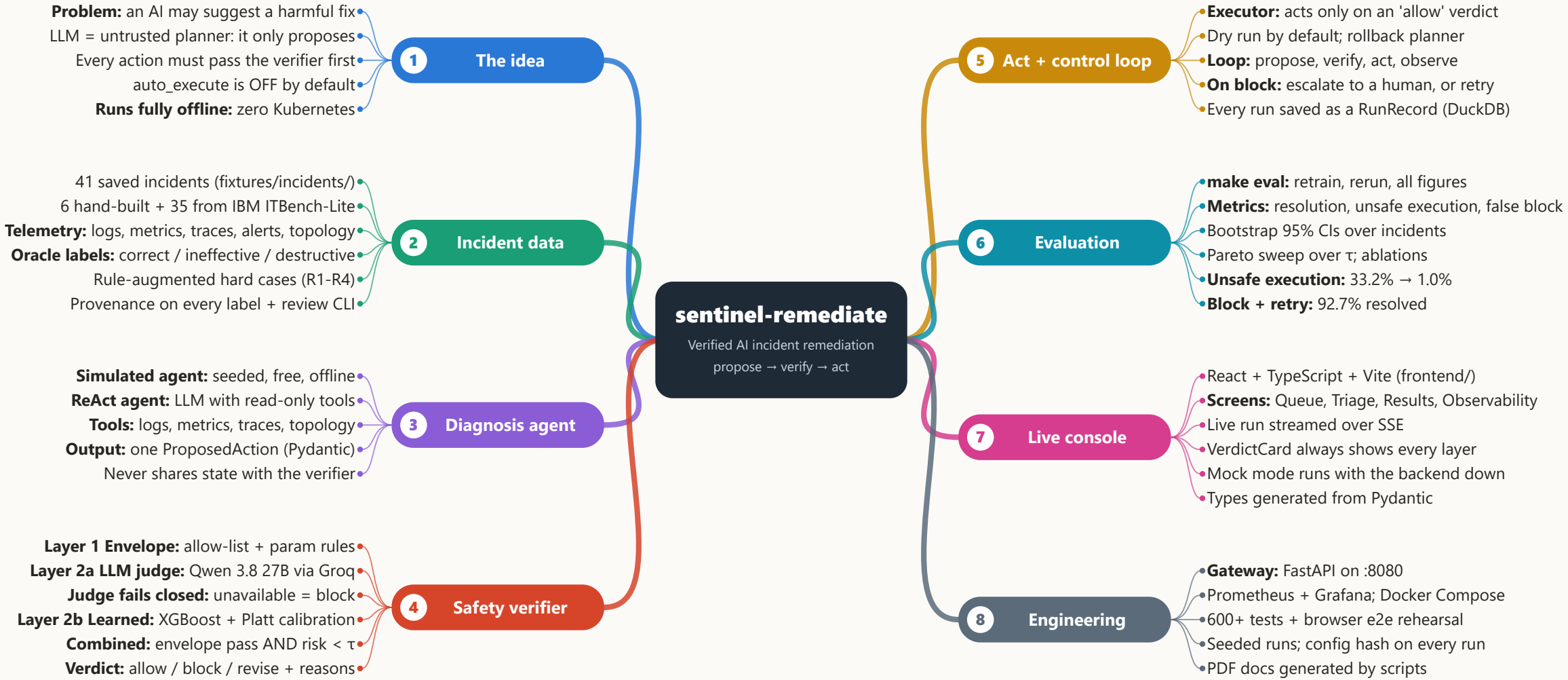
Verified AI Incident Remediation — Project Mind Map

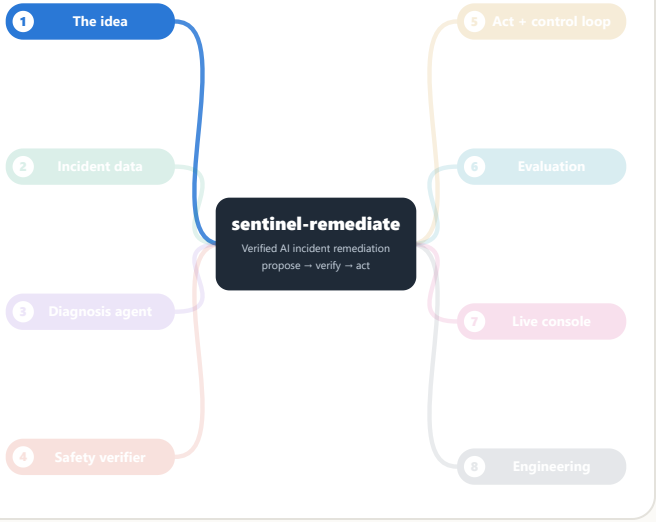
An untrusted AI proposes a fix; a layered verifier decides whether it may run. Left: input and checks · Right: outcome and evidence.



Walk-through order: 1 The idea → 2 Incident data → 3 Diagnosis agent → 4 Safety verifier → 5 Act + control loop → 6 Evaluation → 7 Live console → 8 Engineering

1 The idea

Let an AI propose incident fixes, but never let it act unchecked.



- LLM agents can read logs and metrics and suggest a remediation, confidently, and sometimes wrongly (e.g. scaling down a healthy service).
- Full autonomy is risky; asking a human to approve everything defeats the point.
- Our design treats the LLM as an **untrusted planner**: it may only *propose*. A separate **verifier** decides; only an *allow* reaches the executor.
- Research question: how much unsafe execution can a layered verifier prevent, and what does it cost in false blocks and incidents left unresolved?

33.2%
of runs execute an unsafe action with no verifier

1.0%
with envelope + learned verifier

WHY IT MATTERS

Autonomous operations are only usable if their safety can be enforced and measured, not assumed.

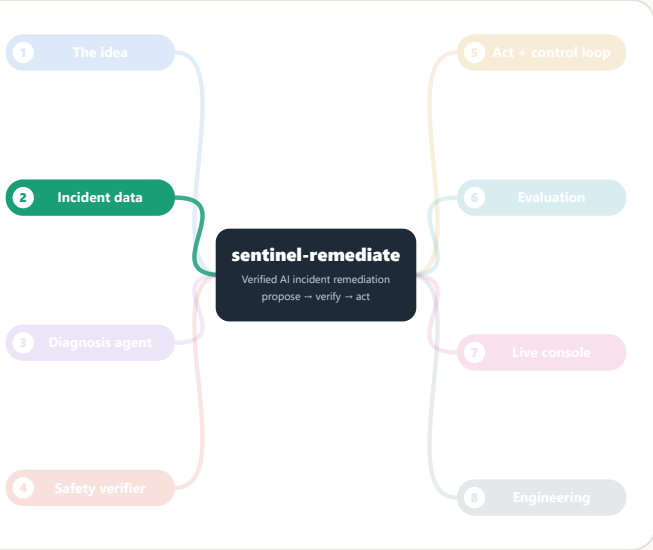
WHERE IN THE CODE

- `CLAUDE.md (golden rules)`
- `src/sentinel/executor/executor.py`

Say to the panel: Our contribution is not a better agent. It is a verifier that makes any agent safer, plus a way to measure exactly how much safer.

2 Incident data

41 reproducible incidents, with an answer key for every candidate action.



- 6 hand-built incidents plus 35 converted from IBM's public **ITBench-Lite** SRE fault scenarios by our converter.
- Each incident carries telemetry (logs, metrics, traces, alerts, service topology) and an **oracle**: candidate actions labeled correct, ineffective, destructive or out-of-envelope.
- 318 labeled actions: 49 correct, 113 ineffective, 115 destructive, 41 out-of-envelope. Every label records its **provenance**.
- Rule augmentation (R1–R4) adds hard within-type contrasts, so a label cannot be guessed from the action type alone.

41
incidents

318
labeled candidate actions

WHY IT MATTERS

Without ground truth there is no safety metric. The ReAct agent never sees the oracle; the learned verifier is trained on it cross-fitted by incident, so it never judges an incident it learned from.

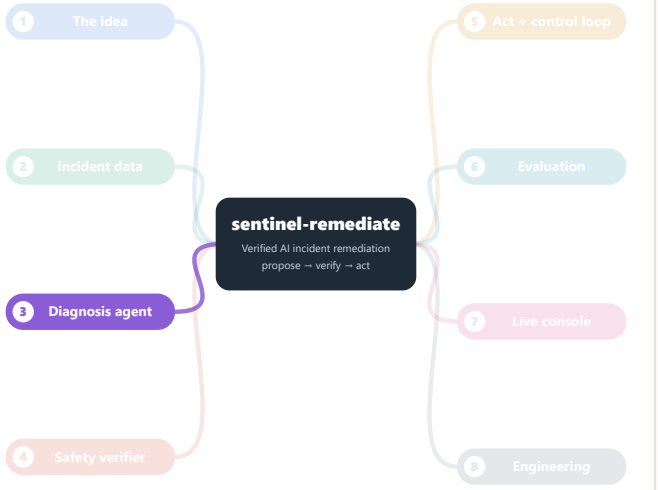
WHERE IN THE CODE

- `fixtures/incidents/`
- `scripts/convert_itbench.py`
- `src/sentinel/labeling/`

Say to the panel: We turned a public benchmark into an offline, labeled testbed, so every experiment is reproducible without a live cluster.

3 Diagnosis agent

Proposes exactly one structured action per step, and is never trusted.



- **ReAct agent**: an LLM reasons step by step and calls read-only tools (logs, metrics, traces, topology), then outputs one `ProposedAction` validated by a schema.
- **Simulated agent**: a seeded, LLM-free stand-in that samples candidate actions at a controlled quality (weak / default / strong). It is a repeatable stimulus for testing the verifier.
- The agent and the verifier never share prompts, clients or state, so the checker cannot be talked into agreeing.

62.9%
unsafe execution, weak agent, no verifier

20.0%
even a strong agent, no verifier

WHY IT MATTERS

Varying agent quality shows the verifier is needed even for a good agent.

WHERE IN THE CODE

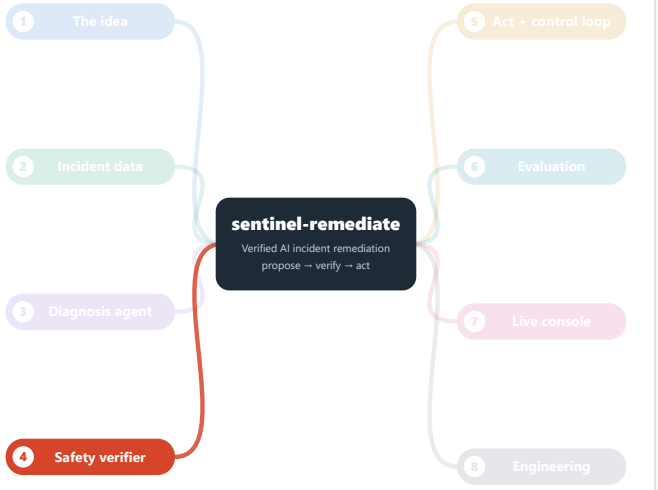
- `src/sentinel/agent/react.py`
- `src/sentinel/agent/simulated.py`
- `src/sentinel/agent/tools.py`

Say to the panel: Even the strong agent executes an unsafe action in about one run in five when nothing checks it. Better agents alone do not solve safety.

BRANCH 4 OF 8

4 Safety verifier

Layered checks: fixed rules first, then a risk estimate compared with a threshold τ.



- **Layer 1 – Envelope**: an allow-list of 10 action types with parameter schemas and a blast-radius cap. Deterministic, no AI.
- **Layer 2a – LLM judge**: an independent model (Qwen 3.8 27B on Groq) returns allow/block/revise, a risk score and reasons. If it is unavailable it *fails closed*.
- **Layer 2b – Learned**: XGBoost on action, incident and topology features; Platt-calibrated; cross-fitted by incident (5 folds). AUC 0.995 on the hard within-type cases (type-only baseline 0.970), ECE 0.033.
- **Policy**: allow only if the envelope passes AND the highest risk is below τ. Every layer's verdict and reasons are recorded and shown.

35.5%
of bad actions caught by the envelope alone

96.9%
caught by envelope + learned (64.6% only by the learned layer)

WHY IT MATTERS

Rules are cheap and certain but narrow; the learned layer catches what rules cannot express.

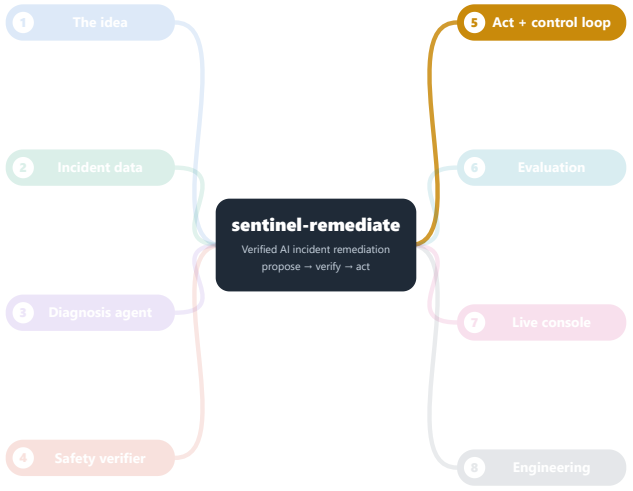
WHERE IN THE CODE

- `src/sentinel/verifier/envelope.py`
- `src/sentinel/verifier/learned.py`
- `src/sentinel/verifier/llm_judge.py`
- `src/sentinel/verifier/policy.py`

Say to the panel: This is the core contribution. The envelope alone stops about a third of bad actions; adding the learned layer stops 96.9%, at a false-block rate of 4.2%.

5 Act + control loop

Only an 'allow' can reach the executor, and every step is recorded.



- The **orchestrator** repeats propose → verify → act → observe until the incident is resolved, escalated, or the step budget runs out.
- **On block**: escalate to a human, or *retry*, feeding the verifier's reasons back to the agent.
- The **executor** dry-runs by default. A real apply needs both the `auto_execute` config flag and an *allow* verdict; reversible actions get a rollback plan.
- Every run is stored as a `RunRecord` in DuckDB with its config hash: no result exists unless it is stored.

92.7%
resolved with block → retry (vs 55.6% with escalate)

2.9%
unsafe execution with retry

WHY IT MATTERS

Blocking alone leaves incidents unresolved; feeding reasons back recovers most of them safely.

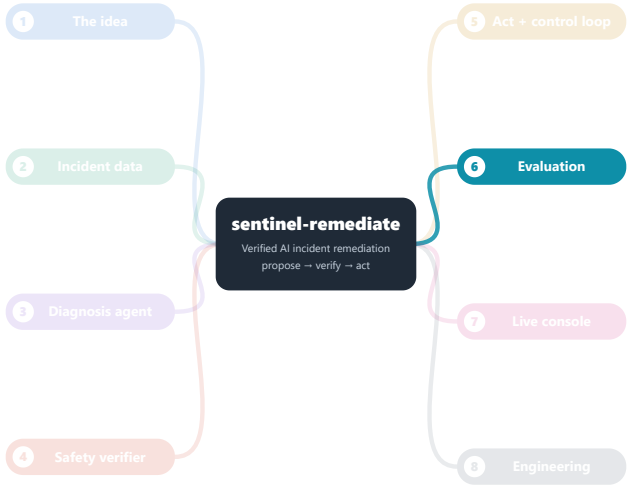
WHERE IN THE CODE

- `src/sentinel/orchestrator/loop.py`
- `src/sentinel/executor/`
- `src/sentinel/store/duckdb_store.py`

Say to the panel: Retry turns the verifier from a brake into a guide: 92.2% of runs end safely resolved, compared with 66.8% with no verifier.

6 Evaluation

7,585 recorded runs, confidence intervals over incidents, one command to regenerate.



- `make eval` retrains the verifier and reruns every configuration on all 41 incidents × 5 seeds, then rebuilds every table and figure from DuckDB.
- Metrics: resolution, bad actions prevented, false blocks, unsafe execution, net-safe resolutions, time to mitigation, cost.
- 95% confidence intervals by bootstrap, resampling *incidents* (not runs), so the uncertainty is honest.
- Ablations: verifier on/off, layers, topology features, block handling, agent quality; Pareto sweep of τ.

7,585
runs recorded

1.0%
unsafe execution (95% CI 0.0%–2.4%)

WHY IT MATTERS

A safety claim needs a trade-off curve and error bars, not a single number.

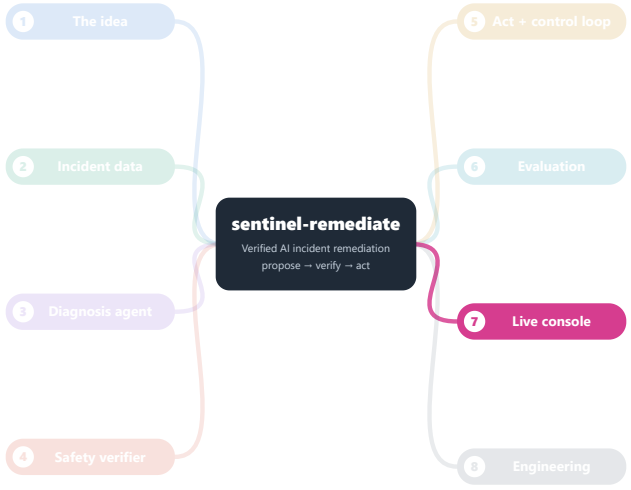
WHERE IN THE CODE

- `src/sentinel/eval/`
- `scripts/run_eval.py`
- `runs/eval/results.md`

Say to the panel: Every figure in the report can be regenerated from scratch with one command, and the same numbers appear in these slides.

7 Live console

Makes the verifier's decisions visible, step by step, as they happen.



- **Queue**: all incidents and their latest outcome. **Triage**: telemetry, topology graph, live loop trace, run controls (verifier, τ).
- The **VerdictCard** always shows every layer (envelope, judge, learned), with inactive ones greyed out, plus the reasons.
- **Results**: Pareto curve, metrics, prevention partition and ablations from DuckDB. **Observability**: Prometheus / Grafana.
- Runs stream live over Server-Sent Events; a mock mode works with the backend down; the TypeScript types are generated from the Python schemas.

4
screens

SSE
live step-by-step streaming

WHY IT MATTERS

Operators trust a verifier they can inspect; the layered view is what the panel will see in the demo.

WHERE IN THE CODE

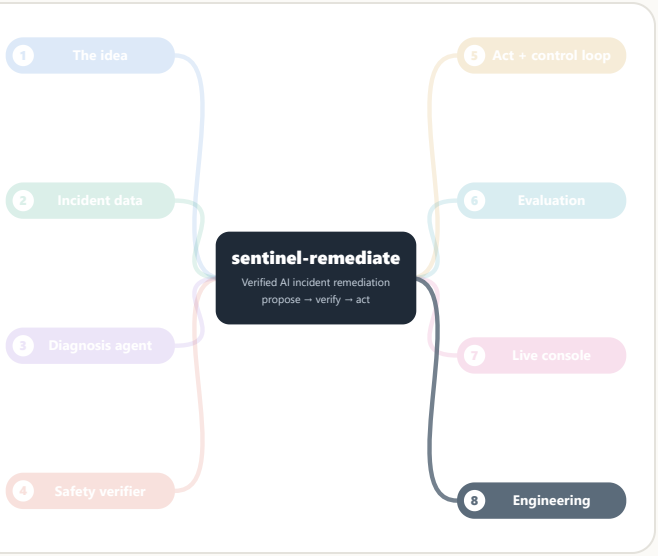
- `frontend/src/routes/`
- `frontend/src/features/triage/VerdictCard.tsx`
- `src/sentinel/gateway/api.py`

Say to the panel: Demo: pick an incident, run it Ungated and then with the verifier, and watch the VerdictCard explain each block.

BRANCH 8 OF 8

8 Engineering

Built to be checked: offline, seeded, tested.



- Runs end to end with **zero Kubernetes**: the environment sits behind an adapter interface, and the offline adapter replays the saved incidents.
- All randomness is seeded from config; every run records its **config hash**.
- **600+ automated tests** plus a headless-browser rehearsal of the live demo (`make e2e`).
- FastAPI gateway, Prometheus/Grafana metrics, Docker Compose; every module boundary is a typed Pydantic model.

0
Kubernetes needed

600+
automated tests

WHY IT MATTERS

Results that cannot be reproduced cannot be trusted; the engineering exists to make the evaluation checkable.

WHERE IN THE CODE

- `Makefile`
- `tests/`
- `docker-compose.yml`
- `src/sentinel/schemas/`

Say to the panel: Anyone can clone the repository, run make test and make eval offline, and get the same numbers.

EVIDENCE

Results at a Glance

7,585 recorded runs · 41 incidents × 5 seeds per configuration · 95% bootstrap CIs over incidents

33.2% → 1.0%
unsafe execution, no verifier → envelope + learned

96.9%
of bad actions caught by envelope + learned

4.2%
false blocks (good actions stopped)

92.7%
resolved with block → retry

CONFIGURATION	UNSAFE EXECUTION	95% CI	RESOLVED	NET-SAFE RESOLVED
No verifier (ungated)	33.2%	26.3%–40.0%	97.6%	66.8%
Envelope only	21.5%	16.1%–26.8%	72.2%	57.6%
Envelope + learned (escalate on block)	1.0%	0.0%–2.4%	55.6%	55.1%
Envelope + learned (retry on block)	**2.9%**	**—**	**92.7%**	**92.2%**

Takeaway: the verifier cuts unsafe execution from 33.2% to 1.0%; with retry it also resolves 92.7% of incidents, and 92.2% end safely resolved versus 66.8% with no verifier.

HONEST SCOPE

Limitations and Next Steps

LIMITATIONS

- The main evaluation uses the **simulated agent**; the LLM ReAct agent and the LLM-judge / combined baselines were not in the full run (API cost and rate limits).
- Most labels are machine-derived: 294 of 318 are flagged for **human review**, which is still pending.
- Incidents are replayed in an **offline environment**, not a live cluster.
- 41 incidents is a small set; generalisation to new fault families is untested.

NEXT STEPS

- Run the LLM-judge and combined baselines in the full evaluation (now possible on the free Groq tier).
- Complete human review of labels with the review CLI.
- Plug in a live-cluster adapter (AIOpsLab / ITBench) behind the same interface.
- Grow the incident set and test on held-out fault families.

Close with: the architecture keeps the AI useful but never in charge, and every claim on these slides is regenerated from stored runs with one command.